Google Skills

The problem: Building intelligence without structure

Introduction

Now that you understand what agents are and why they matter, let's peek under the hood. How does something go from being a text-generating model to an autonomous problem-solver?

In module 1, you learned that agents are defined by five essential characteristics: goal-directed behavior, autonomous operation, proactive initiative, environmental awareness, and tool use. Now here's the interesting part—**all five of these characteristics emerge from just three architectural components working together.**

According to Google's agent whitepaper, every agent is built from these three components: the **model** (thinking), the **tools** (doing), and the **orchestration** (the process connecting them). It's like a car—no matter how fancy the make, they all have an engine, wheels, and a steering system. These three components work together to create the autonomous, goal-directed behavior that defines an agent.

Think of it this way: The model enables autonomous decision-making and environmental awareness. The tools enable real-world action. The orchestration enables goal-directed, proactive behavior. Three components, working together, produce all five agent characteristics.

The problem: Building intelligence without structure

You just learned that agents are built from three architectural components—the model, the tools and the orchestration—these all work together to create the five essential characteristics of an agent. But why do we need this specific architecture? What happens if you try to build an intelligent system without it?

Imagine trying to build a personal assistant AI without these three components properly structured. You have a language model that can understand requests. You have APIs that can do things like send emails, check calendars, and book flights. Surely you can just connect them together?

Here's what happens when you try. A user says: “Schedule a meeting with Sarah next week.” Your system needs to:

1. Understand who Sarah is.
2. Check both calendars for availability.
3. Find a mutually free time.
4. Send a meeting invitation.
5. Add it to both calendars.
6. Confirm with the user.

Without the proper three-component architecture, you end up writing specific code for every possible sequence. What if Sarah is busy? What if the user’s calendar is full? What if the email fails to be sent? Soon you have thousands of lines of brittle if-then-else logic that breaks the moment someone asks something slightly different.

This complexity is real, and it’s a significant challenge. But the root problem isn’t just that the task is complex—it’s that without the model, tools, and orchestration working together as a unified architecture, you're forced to handle that complexity with rigid, brittle code instead of flexible, intelligent behavior. You need those three components properly structured to enable goal-directed, autonomous, and proactive behavior.